AI Learning Roadmap

for a Java Backend Developer

8-week day-by-day plan with projects

Overview

Goal: become an applied AI engineer, someone who builds AI-powered backend systems (LLMs, RAG, agents). You don't need heavy math or PhD-level ML for this, and your Java and Spring skills are a big advantage.

- **Time needed:** 1.5-2 hours per day.
- **Stack:** Java 21, Spring Boot, Spring AI (or LangChain4j), PostgreSQL + pgvector, Ollama (free local models), Docker.
- **How to use this document:** Tick the box in the Done column after each day. Highlighted rows are projects.

Week 1: AI Foundations

Understand what AI, ML and LLMs are and make your first API calls.

Day	Topic	What to learn / do	Done
1	**AI landscape**	Difference between AI, ML, DL and LLMs; generative vs. predictive AI. Note what an AI engineer does compared to an ML engineer.	☐
2	**ML basics**	Supervised vs. unsupervised learning, training vs. inference, features and labels, overfitting. Run one scikit-learn example (Python is fine).	☐
3	**Neural networks intuition**	Neurons, layers, weights and backpropagation at a conceptual level. Watch 3Blue1Brown's neural network series.	☐
4	**Transformers and LLMs**	Tokens, context window, attention (conceptually), why LLMs hallucinate. Try a tokenizer playground.	☐
5	**LLM parameters**	Temperature, top-p, max tokens, system vs. user prompts, pricing per token.	☐
6	**First API calls**	Get an API key (OpenAI, Anthropic or Gemini). Call it with curl, then with Java's HttpClient.	☐
7	**PROJECT 1: CLI chatbot**	Build a Java CLI chatbot with conversation history and a system prompt.	☐

Daily notes: write what you learned, one code snippet, and one open question.

Week 2: Prompt Engineering and LLM Integration

Learn to talk to LLMs reliably and integrate them into Spring Boot.

Day	Topic	What to learn / do	Done
8	**Prompting techniques**	Zero-shot, few-shot, chain-of-thought and role prompts.	☐
9	**Structured output**	Get JSON from the LLM and map it to Java records/POJOs. Practice schema-based prompts.	☐
10	**Spring AI setup**	ChatClient, configuration, switching providers.	☐
11	**Streaming responses**	Use SSE/WebFlux (Flux) to stream tokens to a client.	☐
12	**Chat memory**	Conversation history, token limits, summarizing old messages.	☐
13	**Error handling**	Timeouts, retries, rate limits (429), fallbacks, cost tracking.	☐
14	**PROJECT 2: AI REST API**	Spring Boot service with chat, summarize and classify endpoints. Add streaming and a simple frontend or Postman collection.	☐

Daily notes: write what you learned, one code snippet, and one open question.

Week 3: Embeddings, Vector Databases, RAG Basics

Give the LLM your own data using retrieval-augmented generation.

Day	Topic	What to learn / do	Done
15	**Embeddings**	What a vector is, how semantic similarity works (cosine similarity). Generate embeddings via the API.	☐
16	**Vector databases**	Compare pgvector, Qdrant, Pinecone and Chroma. Run Postgres with pgvector in Docker.	☐
17	**Document ingestion**	Load PDFs/Word files (Apache Tika, Spring AI document readers) and chunk text.	☐
18	**Chunking strategies**	Chunk size, overlap, metadata. Experiment and compare results.	☐
19	**The RAG pipeline**	Full flow: query, embed, retrieve top-k, build prompt, generate.	☐
20	**Citations and grounding**	Return source references and handle "I don't know" answers.	☐
21	**PROJECT 3: Chat with your documents**	Upload PDFs, store embeddings in pgvector, answer questions with sources.	☐

Daily notes: write what you learned, one code snippet, and one open question.

Week 4: Advanced RAG and Evaluation

Make retrieval accurate, secure and measurable.

Day	Topic	What to learn / do	Done
22	**Retrieval quality**	Metadata filtering, MMR, query rewriting.	☐
23	**Hybrid search**	Combine keyword search (BM25 / Postgres full-text) with vector search.	☐
24	**Reranking**	Use cross-encoders or a rerank API to improve results.	☐
25	**Evaluation**	Build a small Q&A test set; measure retrieval accuracy and answer faithfulness. Look at RAGAS concepts.	☐
26	**Multi-tenancy and security**	Per-user document access and permission-based filtering.	☐
27	**Improve Project 3**	Add hybrid search, reranking and an evaluation script.	☐
28	**Review and notes**	Document what worked and what didn't. Revise weak topics.	☐

Daily notes: write what you learned, one code snippet, and one open question.

Week 5: Tool Calling, Agents, MCP

Let the LLM take actions through your backend.

Day	Topic	What to learn / do	Done
29	**Function/tool calling**	How the LLM decides to call your function, with JSON schema definitions.	☐
30	**Tools in Spring AI**	Expose Java methods (@Tool) such as get order status or search products.	☐
31	**Agent concepts**	ReAct loop, planning, and when NOT to use agents (workflows are often better).	☐
32	**Workflow patterns**	Prompt chaining, routing, parallelization, orchestrator-worker.	☐
33	**Model Context Protocol (MCP)**	What MCP is; build a simple MCP server in Java that exposes your service.	☐
34	**Safety for tools**	Human approval for write actions, input validation, least privilege.	☐
35	**PROJECT 4: Customer support agent**	Answers from docs (RAG) and calls your backend APIs for order status, refunds and tickets.	☐

Daily notes: write what you learned, one code snippet, and one open question.

Week 6: Production Engineering (Your Strength)

Apply your backend experience to run AI features reliably.

Day	Topic	What to learn / do	Done
36	**Observability**	Log prompts, responses, latency, tokens and cost with Micrometer/OpenTelemetry. Look at LangSmith/Langfuse.	☐
37	**Caching and cost control**	Semantic caching, prompt caching, smaller models for simple tasks, token budgets.	☐
38	**Guardrails**	Prompt injection, PII masking, output validation, content moderation.	☐
39	**Async and scalable design**	Kafka/queues for batch AI jobs, virtual threads, backpressure.	☐
40	**Testing AI features**	Unit tests with mocked LLMs, golden-set evals, prompt regression tests.	☐
41	**Local models**	Run Ollama (Llama, Mistral, Qwen); learn when private/local beats cloud.	☐
42	**Dockerize and deploy**	docker-compose with app, Postgres and Ollama. Deploy to a cloud VM or Render/Railway.	☐

Daily notes: write what you learned, one code snippet, and one open question.

Week 7: Going Deeper (Optional but Valuable)

Broaden your skills beyond the core stack.

Day	Topic	What to learn / do	Done
43	**Python for AI**	venv, pip, Jupyter, FastAPI. Most AI tutorials use Python, so learn to read it fluently.	☐
44	**Fine-tuning concepts**	Fine-tuning vs. RAG vs. prompting; LoRA and PEFT basics (concepts only).	☐
45	**Multimodal**	Images, audio (speech-to-text), document OCR.	☐
46	**Classic ML serving**	Export an ONNX model and run it in Java (ONNX Runtime / DJL).	☐
47	**Vector search internals**	HNSW and IVF indexes; trade-offs in recall and speed.	☐
48	**Cloud AI services**	AWS Bedrock, Azure OpenAI or Vertex AI.	☐
49	**Reading day**	Anthropic's Building Effective Agents guide and the OpenAI cookbook.	☐

Daily notes: write what you learned, one code snippet, and one open question.

Week 8: Capstone and Portfolio

Build one production-style project and showcase it.

Day	Topic	What to learn / do	Done
50-55	**CAPSTONE: AI Knowledge Assistant**	Multi-user login (Spring Security/JWT); document upload and RAG with hybrid search; agent with tools (tickets, search, email draft); streaming chat UI; observability dashboard (cost and latency); evaluation suite and Docker deployment.	☐
56	**Polish**	Write a README with an architecture diagram, record a demo video, push to GitHub, update resume and LinkedIn.	☐

Daily notes: write what you learned, one code snippet, and one open question.

Project Summary

#	Project	Skills
1	CLI chatbot	API basics
2	AI REST API with streaming	Spring AI, prompts
3	Chat with your PDFs	Embeddings, RAG, pgvector
4	Support agent with tools	Tool calling, agents
5	Capstone knowledge assistant	Everything, production-ready

Tips for Learning

1. Build daily. Spend about 70% of your time coding and 30% reading.
2. Keep a notes file per day with what you learned, one code snippet, and one question you still have.
3. Free resources: Spring AI docs, LangChain4j docs, DeepLearning.AI short courses, 3Blue1Brown, Andrej Karpathy's "Intro to LLMs", and Anthropic/OpenAI docs.
4. Skip heavy math at first. Come back to linear algebra and statistics only if you want to move toward ML engineering.